

对编程语言理论中元语言和目标语言概念的辨析——面向编程语言理论初学者

Nicolas H. Zhong

由于常见教科书上对元语言 (metalinguage) 和目标语言 (object language) 的区分常常不够强调, 以及初次接触形式化证明这一概念带来的陌生感, 编程语言理论的初学者经常不够熟悉这两者之间的边界, 进而产生一些困惑, 比如:

- 教材上说推理规则 (inference rule) 中的长横线是 “if $\langle \text{premises} \rangle$ then $\langle \text{conclusion} \rangle$ ” 的意思, 那么自然语言中的 “if-then” 的严格定义又是什么?
- 进一步, 这个长横线和正在被定义的编程语言中的 if 语句有什么区别; 判断 (judgement) 和正在被定义的编程语言中的 bool 值又有什么区别?
- 什么能被形式化, 什么不能被形式化? 任何概念都能被形式化吗?

出现这种困惑, 很大程度上是因为常用的编程语言理论入门书并没有花足够的笔墨解释这个问题。[Pie02, Chapter 3] 对此有稍多的强调, 即

The symbol t in the right-hand sides of the rules of this grammar is called a *metavariable*. It is a variable in the sense that it is a place-holder for some particular term, and “meta” in the sense that it is not a variable of the *object language*—the simple programming language whose syntax we are currently describing—but rather of the *metalinguage*—the notation in which the description is given.

但基本也尽于此。而 [Har16] 和 [Pie+22] 对此几乎没有解释。很多数理逻辑入门教材倒是有更多阐述, 但是想入门编程语言理论其实又未必要先系统读一遍一阶逻辑。

本文将集中而直观地解释和辨析元语言和目标语言的概念, 并且将一些常引起疑惑的说法作为例子进行解释。本文不打算严格地介绍一些逻辑概念, 也不会详细解释归纳定义等概念 (关于这些, 看任何一本标准教材, 比如 [Har16]), 只是传达一种直观理解, 澄清元语言和目标语言之间的边界, 让自学编程语言理论的初学者不至于陷入一些微妙的文字辨析。

1 元语言和目标语言

元语言和目标语言的概念是相对的, 不是说某个语言永远是元语言、某个语言永远是目标语言。比如, 当用 Rocq 形式化某个编程语言的定义并证明其一些性质 (所谓元理论, metatheory) 的时候, Rocq 是元语言; 而当我们定义 Rocq 这个语言本身的时候, 它又是目标语言了。我们平时直接说 “元语言” 和 “目标语言”, 是默认了上下文中隐含了这种的相对性。

这样一来很容易看出，当你要定义和研究一个编程语言 L ，你在“定义”和“描述”它时用的语言就是这时的元语言，比如英语、元变量和长横线（见上文对 [Pie02, Chapter 3] 的引用及后文例子），或者某个定理证明器。而 L 在这个时候就是目标语言。

在这种情况下，目标语言 L 就是被所谓“形式化”的那个。那么元语言可不可以被“形式化”呢？可以，但要用另一个“元元语言”，而这时你又会问这个“元元语言”可不可以被形式化。然而根据 Tarski's undefinability theorem，任何一个（有一定强度的）形式系统都不能形式定义它自身，所以这个过程是不能穷尽的。通常在研究编程语言时，我们来到“元语言”这一步就好了（这时的元语言一般为自然语言加上日常的数学概念），而不进一步形式化“元语言”。而元语言的语义，则是大家日常生活中公认的理解，比如自然语言中一些逻辑关系的含义，或者一些基本数学概念（如整数、树、序关系等）。

可以提及的一点是，元语言的表达能力并不需要比被它描述的目标语言更“强”。比如以一阶逻辑 + ZFC 集合论作为元语言，可以定义和研究看起来更“强”的依值类型论。

2 一些例子

本节列举一些常见的例子，并说明它们是元语言还是目标语言。我们用一个非常简单的语言作为贯穿本节的例子。

语法：

Type	τ	::=	int	integers
Expression	e	::=	num[n]	integer literal
			plus(e_1, e_2)	addition
			let $x := e_1$ in e_2	local binding

定型 (typing) 规则：

$$\frac{}{\Gamma, x : \tau \vdash x : \tau} \quad \frac{}{\Gamma \vdash \text{num}[n] : \text{int}} \quad \frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash \text{plus}(e_1, e_2) : \text{int}} \quad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x := e_1 \text{ in } e_2 : \tau_2}$$

操作语义 (operational semantics)：

$$\frac{}{\text{num}[n] \text{ val}} \quad \frac{n_1 + n_2 = n}{\text{plus}(\text{num}[n_1], \text{num}[n_2]) \rightarrow \text{num}[n]} \quad \frac{n \text{ val}}{\text{let } x := e_1 \text{ in } e_2 \rightarrow [e_1/x]e_2}$$

这里的目标语言，就是上面的语法所描述的那样。一些目标语言代码包括 `num[3]`, `plus(num[1], num[2])`，或者 `let  $x := \text{num}[5]$  in plus(num[3],  $x$ )`。

下列要素属于元语言：

- Backus-Naur form 本身，即上述用于定义目标语言语法的 `::=`、`|` 等记号，而不是目标语言自己的语法。
- 判断 (judgement) 和判断形式 (judgement form)：由定型规则定义的定性判断形式 $\cdot \vdash \cdot : \cdot$ ，以及将其应用到具体的语法元素之后形成的判断，如 $[x : \text{int}] \vdash \text{plus}(x, \text{num}[3]) : \text{int}$ 。
- 表述操作语义的判断形式 `val` 和 $\rightarrow$ 。Transition 本身确乎不是目标语言内部的要素，而是目标语言的元性质。

- 推理规则中的长横线，它们就是自然语言中的“if ... then ...”，只不过这里用符号写比较清晰简洁。
- 关于这个语言的元性质和它们的证明，比如在该语言的语法结构上使用结构归纳法（structural induction）；又或者比如在定型或者语义规则的推导树（derivation tree）上做结构归纳，以证明该语言的一些元性质，如类型安全（type safety）[Har16, Chapter 4–6]。特别注意，这里的归纳法是元语言中的归纳法，归纳法所基于的“语法树/导出树这一结构的良基性（well-foundedness）”也是元语言中的数学性质（类似地，你还可以有“元语言中的整数”等概念）。这些元语言中的数学性质（至少在这里）没有被形式地定义，而是根据我们日常理解的数学和逻辑。我想这个例子应该很清晰地展示了元语言和目标语言的边界，毕竟在这里这个这么简单的目标语言之内部，实在是不可能表达出类似于“它的类型安全性”这样的判断和推理，因而很自然这些全都是“元性质”。

参考资料

- [Har16] Robert Harper. *Practical Foundations for Programming Languages*. 2nd. USA: Cambridge University Press, 2016. ISBN: 1107150302.
- [Pie+22] Benjamin C. Pierce et al. *Software Foundations*. 2022. URL: <https://softwarefoundations.cis.upenn.edu/>.
- [Pie02] Benjamin C. Pierce. *Types and Programming Languages*. 1st. The MIT Press, 2002. ISBN: 0262162091.